

1) The Task

You are given 5 MongoDB collections:

- customers
- orders
- order_items
- payments
- products

we need you to build one aggregation pipeline that returns an analytics summary by customer country for the specific given period:

- From = 2025-10-01T00:00:00.000+02:00 (Africa/Cairo)
- To = 2026-01-01T00:00:00.000+02:00 (Africa/Cairo)

Business Rules (important)

1. Only include orders where status = "completed" and placedAt is within the period.
2. Payments:
 - Consider only payments.status = "succeeded"
 - Ignore any payment where provider = "test"
 - An order may have multiple charges and multiple refunds
 - Order Net Revenue = sum(charges) - sum(refunds) (only for included completed orders)
3. Allocate net revenue to categories:
 - Each order has item extended prices (qty * unitPrice)
 - CategoryNetRevenue is the sum of each item's share of order net revenue:
 - $\text{itemNet} = \text{orderNetRevenue} * (\text{itemExtPrice} / \text{orderGrossItems})$
4. Cost & margin:
 - COGS is computed from products: sum(qty * unitCost) for included completed orders
 - GrossMargin = NetRevenue - COGS
5. Output per country:
 - country
 - ordersCount
 - uniqueCustomers
 - newCustomers (distinct customers whose createdAt falls in the same period)
 - returningCustomers (distinct customers who have ≥ 2 completed orders in the period)
 - netRevenue
 - cogs
 - grossMargin
 - aov (average order value) = netRevenue / ordersCount
 - topCategoryByNetRevenue (single category name)

Sort output by netRevenue desc.

2) Mongoose Models (NestJS)

customers

```
// customers.schema.ts
{
  _id: ObjectId,
  fullName: string,
  country: 'EG' | 'UAE',
  createdAt: Date
}
```

products

```
// products.schema.ts
{
  _id: ObjectId,
  sku: string,
  name: string,
  category: 'Tickets' | 'Merch' | 'Subscriptions',
  unitCost: number
}
```

orders

```
// orders.schema.ts
{
  _id: ObjectId,
  customerId: ObjectId, // ref customers
  status: 'completed' | 'canceled',
  currency: 'USD',
  placedAt: Date
}
```

order_items

```
// order-items.schema.ts
{
  _id: ObjectId,
  orderId: ObjectId, // ref orders
  productId: ObjectId, // ref products
  qty: number,
  unitPrice: number
}
```

payments

// payments.schema.ts

```
{
  _id: ObjectId,
  orderId: ObjectId, // ref orders
  kind: 'charge' | 'refund',
  amount: number,
  provider: 'stripe' | 'paypal' | 'test',
  status: 'succeeded' | 'failed',
  paidAt: Date
}
```

3) Seed Data (exact inserts)

Use Mongo shell / mongosh. (IDs are fixed to make expected output deterministic.)

// ===== customers =====

```
db.customers.insertMany([
  { _id: ObjectId("65600000000000000000000001"), fullName: "Ahmed Ali", country: "EG",
    createdAt: ISODate("2025-09-20T10:00:00.000Z") },
  { _id: ObjectId("65600000000000000000000002"), fullName: "Mona Saad", country:
    "EG", createdAt: ISODate("2025-11-05T12:00:00.000Z") },
  { _id: ObjectId("65600000000000000000000003"), fullName: "Sara Noor", country:
    "UAE", createdAt: ISODate("2025-10-10T09:00:00.000Z") },
  { _id: ObjectId("65600000000000000000000004"), fullName: "Omar Hassan", country:
    "UAE", createdAt: ISODate("2025-08-01T08:00:00.000Z") }
]);
```

// ===== products =====

```
db.products.insertMany([
  { _id: ObjectId("65700000000000000000000001"), sku: "TICK-STD", name: "Standard
    Ticket", category: "Tickets", unitCost: 60 },
  { _id: ObjectId("65700000000000000000000002"), sku: "MERCH-TS", name: "T-Shirt",
    category: "Merch", unitCost: 20 },
  { _id: ObjectId("65700000000000000000000003"), sku: "SUB-MON", name: "Monthly
    Sub", category: "Subscriptions", unitCost: 150 }
]);
```

// ===== orders =====

```
db.orders.insertMany([
  { _id: ObjectId("65800000000000000000000001"), customerId:
    ObjectId("65600000000000000000000001"), status: "completed", currency: "USD",
    placedAt: ISODate("2025-10-02T08:00:00.000Z") },
  { _id: ObjectId("65800000000000000000000002"), customerId:
    ObjectId("65600000000000000000000001"), status: "completed", currency: "USD",
    placedAt: ISODate("2025-11-15T14:00:00.000Z") },
]);
```

```
{_id: ObjectId("658000000000000000000003"), customerId:
ObjectId("65600000000000000000000002"), status: "completed", currency: "USD",
placedAt: ISODate("2025-12-01T10:30:00.000Z") },
```

```
// canceled order (must be excluded even if it has succeeded payments)
{ _id: ObjectId("65800000000000000000000004"), customerId:
ObjectId("65600000000000000000000003"), status: "canceled", currency: "USD",
placedAt: ISODate("2025-10-20T11:00:00.000Z") },
```

```
{ _id: ObjectId("65800000000000000000000005"), customerId:
ObjectId("65600000000000000000000004"), status: "completed", currency: "USD",
placedAt: ISODate("2025-10-25T16:00:00.000Z") },
{ _id: ObjectId("65800000000000000000000006"), customerId:
ObjectId("65600000000000000000000003"), status: "completed", currency: "USD",
placedAt: ISODate("2025-12-20T18:00:00.000Z") }
]);
```

```
// ===== order_items =====
```

```
db.order_items.insertMany([
  // O1 (Ahmed): Tickets 2*100, Merch 1*50 => gross 250
  { _id: ObjectId("65900000000000000000000001"), orderId:
  ObjectId("65800000000000000000000001"), productId:
  ObjectId("65700000000000000000000001"), qty: 2, unitPrice: 100 },
  { _id: ObjectId("65900000000000000000000002"), orderId:
  ObjectId("65800000000000000000000001"), productId:
  ObjectId("65700000000000000000000002"), qty: 1, unitPrice: 50 },
```

```
  // O2 (Ahmed): Tickets 1*130 => gross 130
  { _id: ObjectId("65900000000000000000000003"), orderId:
  ObjectId("65800000000000000000000002"), productId:
  ObjectId("65700000000000000000000001"), qty: 1, unitPrice: 130 },
```

```
  // O3 (Mona): Subscription 1*300 => gross 300
  { _id: ObjectId("65900000000000000000000004"), orderId:
  ObjectId("65800000000000000000000003"), productId:
  ObjectId("65700000000000000000000003"), qty: 1, unitPrice: 300 },
```

```
  // O4 canceled (Sara): Tickets 1*100 => gross 100 (excluded)
  { _id: ObjectId("65900000000000000000000005"), orderId:
  ObjectId("65800000000000000000000004"), productId:
  ObjectId("65700000000000000000000001"), qty: 1, unitPrice: 100 },
```

```
  // O5 (Omar): Merch 4*40 => gross 160
  { _id: ObjectId("65900000000000000000000006"), orderId:
  ObjectId("65800000000000000000000005"), productId:
  ObjectId("65700000000000000000000002"), qty: 4, unitPrice: 40 },
```

```

// O6 (Sara): Tickets 3*90 => gross 270
{ _id: ObjectId("65900000000000000000000007"), orderId:
ObjectId("65800000000000000000000006"), productId:
ObjectId("65700000000000000000000001"), qty: 3, unitPrice: 90 }
]);

// ===== payments =====
db.payments.insertMany([
  // O1: charge 250
  { _id: ObjectId("65a00000000000000000000001"), orderId:
ObjectId("65800000000000000000000001"), kind: "charge", amount: 250, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-10-02T08:05:00.000Z") },

  // O2: charge 130, refund 20 => net 110
  { _id: ObjectId("65a00000000000000000000002"), orderId:
ObjectId("65800000000000000000000002"), kind: "charge", amount: 130, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-11-15T14:05:00.000Z") },
  { _id: ObjectId("65a00000000000000000000003"), orderId:
ObjectId("65800000000000000000000002"), kind: "refund", amount: 20, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-11-16T09:00:00.000Z") },

  // O3: charge 300
  { _id: ObjectId("65a00000000000000000000004"), orderId:
ObjectId("65800000000000000000000003"), kind: "charge", amount: 300, provider:
"paypal", status: "succeeded", paidAt: ISODate("2025-12-01T10:40:00.000Z") },

  // O4 canceled: has succeeded charge (must be ignored due to order status)
  { _id: ObjectId("65a00000000000000000000005"), orderId:
ObjectId("65800000000000000000000004"), kind: "charge", amount: 100, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-10-20T11:10:00.000Z") },

  // O5: charge 160, plus a "test" payment that must be ignored
  { _id: ObjectId("65a00000000000000000000006"), orderId:
ObjectId("65800000000000000000000005"), kind: "charge", amount: 160, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-10-25T16:05:00.000Z") },
  { _id: ObjectId("65a00000000000000000000007"), orderId:
ObjectId("65800000000000000000000005"), kind: "charge", amount: 999, provider: "test",
status: "succeeded", paidAt: ISODate("2025-10-25T16:06:00.000Z") },

  // O6: charge 270, plus a failed payment that must be ignored
  { _id: ObjectId("65a00000000000000000000008"), orderId:
ObjectId("65800000000000000000000006"), kind: "charge", amount: 270, provider:
"stripe", status: "succeeded", paidAt: ISODate("2025-12-20T18:05:00.000Z") },

```

```
{_id: ObjectId("65a000000000000000000009"), orderId:
ObjectId("65800000000000000000000006"), kind: "charge", amount: 270, provider:
"stripe", status: "failed",  paidAt: ISODate("2025-12-20T18:04:00.000Z") }
]);
```

4) Expected Output

```
[
  {
    "country": "EG",
    "ordersCount": 3,
    "uniqueCustomers": 2,
    "newCustomers": 1,
    "returningCustomers": 1,
    "netRevenue": 660,
    "cogs": 350,
    "grossMargin": 310,
    "aov": 220,
    "topCategoryByNetRevenue": "Tickets"
  },
  {
    "country": "UAE",
    "ordersCount": 2,
    "uniqueCustomers": 2,
    "newCustomers": 1,
    "returningCustomers": 0,
    "netRevenue": 430,
    "cogs": 260,
    "grossMargin": 170,
    "aov": 215,
    "topCategoryByNetRevenue": "Tickets"
  }
]
```